

2026(전기) 캡스톤 디자인 착수 보고서

Agentic AI 기반 클라우드 비용 이상 징후 탐지 및 자율 복구 시스템



Detection

소속	부산대학교 정보컴퓨터공학부
팀명	Detection (Team.27)
팀원	202355594 허소영 (thdual429@pusan.ac.kr) 202355514 강지원 (kjlw497@pusan.ac.kr) 202355540 박소영 (qkrthdud1309@pusan.ac.kr)

목차

1. 연구 주제
2. 과제 배경 및 목표
 - 2.1 대상 문제
 - 2.2 요구조건 분석
 - 2.2.1 이상 탐지 요구사항
 - 2.2.2 분류 및 추론 요구사항
 - 2.2.3 복구 실행 요구사항
 - 2.2.4 안전 요구사항
 - 2.2.5 확장성 및 구조적 요구사항
 - 2.3 연구 목표
 - 2.4 배경 지식 (포함 미정)
3. 시스템 구성
 - 3.1 시스템 구성도
 - 3.1.1 데이터 수집 및 버퍼링
 - 3.1.2 Detection Agent
 - 3.1.3 Classification Agent
 - 3.1.4 Decision Agent
 - 3.1.5 Action Agent
 - 3.1.6 QA Agent
 - 3.1.7 Logging Agent
 - 3.2 시스템 개발 환경
 - 3.3 방법 구체화
 - 3.3.1 Detection Agent
 - 3.3.2 Classification Agent
 - 3.3.3 Decision Agent
 - 3.3.4 Action Agent
 - 3.3.5 QA Agent
 - 3.3.6 Logging Agent
4. 현실적 제약 상황 및 대책
 - 4.1 LLM 환각(Hallucination)으로 인한 오판
 - 4.2 AWS API 및 데이터 수집 관련 제약
 - 4.3 일부 액션의 비가역성
5. 요약 및 기대 효과
 - 5.1 결론
 - 5.2 기대 효과
6. 개발 일정 및 역할 분담
 - 6.1 개발 일정
 - 6.2 역할 분담

1. 연구 주제

주제 : Agentic AI 기반 클라우드 비용 이상 징후 탐지 및 자율 복구 시스템

본 프로젝트는 클라우드 재무 운영 방법론인 FinOps(Financial Operations) 관점에서, Agentic AI 아키텍처를 활용하여 클라우드 환경에서 발생하는 비용 이상 징후를 실시간으로 탐지하고 자율적으로 복구하는 운영 시스템을 설계 및 구현하는 것을 목표로 한다. 5개의 단계로 동작하는 AI 에이전트(Detection, Classification, Decision, Action, QA)가 제안하는 기술의 전체 파이프라인을 구성하며, 통계적 이상 탐지(Z-score)와 머신러닝 기반 탐지(Isolation Forest)를 앙상블로 결합하여 비용 이상 징후 탐지 정확도를 향상시키고 LLM 추론을 통해 최적의 복구 액션을 자율적으로 판단 및 실행한다.

2. 과제 배경 및 목표

2.1 대상 문제

클라우드 서비스의 확산에 따라 기업의 인프라 운영 방식은 온프레미스 환경에서 클라우드 중심 환경으로 빠르게 전환되고 있다. 이에 따라 기업의 클라우드 사용 비용 또한 지속적으로 증가하고 있으며, 과다하거나 비정상적인 비용이 발생하지 않도록 관리하는 것이 중요한 운영 과제가 되고 있다. 이러한 배경에서 등장한 개념이 바로 FinOps이다.

FinOps는 엔지니어링, 재무, 비즈니스 팀이 협력하여 클라우드 비용의 가시성을 확보하고 지출을 최적화하는 것을 핵심 가치로 삼는다. 특히 사용량 기반 과금(Pay-as-you-go) 구조를 사용하는 클라우드 환경에서는 단시간의 비정상적인 리소스 사용이나 공격 트래픽이 대규모 비용 손실로 이어질 수 있기 때문에, 실시간 비용 모니터링과 자동 대응 체계가 핵심 과제가 되고 있다.

그러나 현재의 클라우드 비용 이상 탐지 시스템은 임계치 기반의 단순 알람 기능에 그치고 있으며, 이상 상황의 원인 분석 및 복구 조치는 여전히 관리자에게 의존하고 있어 FinOps가 지향하는 비용 최적화 및 운영 자동화를 충분히 실현하지 못하고 있다.

기존 방식의 주요 한계점은 다음과 같다.

- 비용 이상 원인의 다양성: Idle 리소스, 트래픽 폭증, DDoS/EDoS 공격 등이 혼재되어 정확한 상황 파악이 어려우며, 정해진 규칙 기반 대응 방식으로는 한계가 있다.
- 수동 대응의 비효율: 엔지니어가 클라우드 비용 이상을 감지하고 조치를 취하기까지 많은 시간이 소요되어, 불필요한 비용이 지속적으로 발생한다.

기존 한계를 극복하기 위해 본 프로젝트는 다음과 같은 차별화된 기능을 제안한다.

- 단순 탐지에 그치지 않고 Detect - Think - Act까지의 전 과정을 자동화하여 비용 이상 현상에 즉각 대응하는 Fully-Automated End-to-End 파이프라인을 설계한다.
- LLM 기반 Agentic AI를 활용함으로써 추론 과정을 블랙박스도 남기지 않고 의사결정의 근거와 흐름을 설명 가능한 형태의 화이트박스로 제공한다.

- EDoS, DDoS와 같이 기존 솔루션이 대응하기 어려웠던 고도화된 보안 위협까지 효과적으로 대응함으로써 클라우드 자원의 안정성과 서비스 운영 신뢰성을 향상시키는 것을 목표로 한다.

이를 실현하기 위한 핵심 설계 전략은 다음과 같다.

1. 이상불 탐지

Z-score는 단일 지표의 단기 스파이크에 민감하고, Isolation Forest는 다변량 복합 드리프트를 잘 포착한다. 두 알고리즘을 OR로 결합함으로써 False Negative를 최소화하고, False Positive는 Classification Agent의 LLM 판단으로 필터링하는 2단계 구조를 채택했다.

2. Rule-based + LLM 하이브리드 진단

모든 이상 징후를 LLM에 위임하면 API 비용과 응답 지연이 발생한다. 명확한 케이스는 규칙으로 처리하고, 모호한 케이스만 LLM에 위임함으로써 효율과 정확도를 동시에 달성한다.

3. 자율성과 안전성의 균형

Risk LOW 액션은 즉시 실행하되, Risk MED/HIGH 액션은 60분 간 관리자 승인 대기 후 처리된다. QA 검증 실패 시, 최대 2회 롤백을 보장함으로써 자율 복구의 안전성을 확보한다.

2.2. 요구조건 분석

2.2.1 이상 탐지 요구사항

- 클라우드 리소스(EC2, Lambda, S3, Auto Scaling)의 비용 및 성능 메트릭을 실시간으로 수집하여 이상 여부를 판단한다.
- 탐지 알고리즘은 Z-Score와 Isolation Forest를 활용하며, 데이터는 AWS Cost Explorer API 및 CloudWatch를 통해 수집한다.
- 탐지된 이상은 다음과 같이 정의된 세 가지 패턴으로 분류한다.
 - 낭비형(Cost Inefficiency): 긴급도가 낮고 안전하게 자동처리 가능한 경우로, 대표적인 예로 좀비 리소스와 오버프로비저닝이 있다.
 - 급증형(Cost Spike) : 긴급도가 높고 처리 위험도가 높은 경우로, 대표적인 예로 트래픽 폭증, Lambda 호출 폭증 등이 있다.
 - 위험형(Risk/Security) : 매우 민감하여 보수적으로 처리해야하는 경우로, 대표적인 예로 EDoS, 비정상적 접근이 있다.

2.2.2 분류 및 추론 요구사항

- 탐지된 이상 징후는 LLM 에이전트가 유형을 분류하고 상황에 맞는 복구 액션을 결정한다.

- 추론 결과뿐만 아니라 판단 근거와 추론 과정이 단계별로 기록되어 설명 가능한 형태로 제공되어야 한다.
- LangChain 및 LangGraph를 활용하여 에이전트 간 흐름을 관리하며, 분류·판단·QA 단계가 명확히 분리된 구조를 가져야 한다.

2.2.3 복구 실행 요구사항

- 리소스 유형과 이상 패턴에 따라 사전 정의된 복구 액션이 자동으로 실행되어야 한다.
 - EC2: 좀비 인스턴스 중지 / 오버프로비저닝 타입 변경 / 과부하 시 스케일아웃 수행
 - Lambda: 무한루프 및 호출 폭증 시 동시성 제한 적용
 - S3: 데이터 유출 의심 시 퍼블릭 접근 차단 / 트래픽 폭증 시 IP 기반 버킷 정책 적용
 - Auto Scaling: EDoS 의심 시 최대 인스턴스 수 제한
- 모든 액션은 Boto3(AWS SDK)를 통해 실행한다.

2.2.4 안전 요구사항

- 위험형 이상의 경우 서비스 장애 방지를 위해 보수적으로 동작해야 한다.
- 위험도가 높은 복구 액션 실행 전 사용자에게 알림을 전송하고, 확인이 없을 경우 60분 유예 후 자동 실행한다.
- QA 에이전트는 액션 실행 전 SLA 기준으로 사전 검증을 수행하며, 비용 감소율 대비 시스템 품질을 함께 판단한다.

2.2.5 확장성 및 구조적 요구사항

- 하드코딩 없이 확장 가능한 구조로 설계한다.
- 전체 파이프라인은 Dockerfile로 컨테이너화하여 재현성과 확장성을 확보한다.
- 초기 rule-based 로직 이후 비지도 학습 기반 클러스터링으로 단계적으로 발전시킨다
- LLM 모델은 Gemini와 Ollama를 우선 적용하며, 추후 다른 모델 교체에 있어서 유연해야 한다.

2.3 연구 목표

본 연구는 AWS 클라우드 환경에서 발생하는 비용 이상 및 보안 위협을 자동으로 탐지·분석·복구하는 LLM 기반 자율 에이전트 시스템을 설계 및 구현하는 것을 목표로 한다. Z-Score와 Isolation Forest를 결합한 이중 탐지 레이어로 이상 징후를 식별하고, LangChain ReAct 에이전트가 이를 분류·판단하여 Boto3를 통해 AWS 리소스를 직접 제어하는 Detect - Think - Act 자율 복구 파이프라인을 구축한다. 나아가 EDoS·DDoS 등 고도화된 보안 위협까지 대응 범위를 확장하고, 에이전트의 모든 추론 과정을 로그로 기록하여 의사결정의 설명 가능성을 확보한다. 최종적으로 시스템의 비용 절감 효과를 정량적 실험을 통해 검증함으로써 실용적 가치를 객관적으로 입증한다.

2.4 배경 지식

Agentic AI 및 LangGraph

Agentic AI는 단일 LLM 프롬프트를 넘어, 다수의 전문화된 에이전트가 상태를 공유하며 협력적으로 복잡한 작업을 수행하는 패러다임이다. LangGraph는 LangChain 위에서 동작하는 상태 머신 프레임워크로, 각 에이전트를 그래프 노드로 정의하고 조건부 엣지로 제어 흐름을 관리한다. ReAct(Reason + Act) 패턴을 통해 LLM이 추론과 행동을 반복적으로 수행하며 목표를 달성한다.

이상 탐지 알고리즘

이상 탐지 분야의 주요 기법은 단일 지표 기반 통계 방법과 다변량 머신러닝 방법으로 구분된다.

- Z-score 기반 탐지: 슬라이딩 윈도우 내 평균과 표준편차를 계산하여 현재 값이 $\mu \pm k\sigma$ 범위를 벗어나면 이상으로 판정한다. 구현이 단순하고 실시간 처리에 적합하나, 단일 지표에만 적용 가능하다는 한계가 있다.
- Isolation Forest: 랜덤 파티셔닝을 통해 이상 데이터를 빠르게 격리하는 앙상블 방법론이다. 다변량 피쳐 벡터에 적용 가능하며, 훈련 데이터에 레이블이 불필요한 비지도 학습 방식이다. 이상 점수는 평균 격리 깊이를 기반으로 계산된다.

클라우드 비용 관리 (FinOps)

FinOps는 클라우드 비용의 가시성·최적화·통제를 목표로 하는 운영 프레임워크이다. AWS는 Cost Explorer, CloudWatch, Trusted Advisor 등의 네이티브 도구를 제공하나, 이들은 알림 및 시각화에 그치며 자율적 복구 행동은 지원하지 않는다. 본 연구는 Boto3 SDK를 통해 자율 복구 행동까지 수행함으로써 기존 FinOps 도구의 한계를 극복한다.

로컬 및 클라우드 기반 LLM 활용

Ollama는 macOS, Linux, Windows 환경에서 오픈소스 LLM을 로컬로 실행할 수 있는 플랫폼으로, 외부 API 호출 없이 데이터 프라이버시를 유지하면서 LLM 추론이 가능하다. OpenAI 호환 API 엔드포인트를 제공하기 때문에 LangChain과의 통합이 용이하며, 추가 비용 없이 다양한 오픈소스 모델을 로컬 환경에서 자유롭게 운용할 수 있다. 본 프로젝트에서는 Ollama를 통해 제공되는 Qwen, Llama, SuperGemma 계열 등의 다양한 오픈소스 모델을 적용하고, 모델 크기 대비 추론 성능 및 실행 시간을 비교 분석하여 시스템 환경에 가장 적합한 모델을 선정하고자 한다.

Gemini API는 무료로 제공되는 LLM API로, 상대적으로 가볍고 빠른 응답 속도를 제공하여 프로토타입 개발 및 실험 단계에서 효율적으로 활용할 수 있다. 본 프로젝트에서는 Ollama 기반 로컬 모델과 Gemini API 기반 모델 간의 응답 속도 및 추론 성능을 비교함으로써, 비용 효율성과 실시간성을 고려한 최적의 LLM 운용 방안을 도출하고자 한다.

3. 시스템 구성

3.1. 시스템 구성도

본 시스템은 데이터 수집, 이상 탐지, 진단, 의사결정, 행동, 검증, 로깅의 7단계로 구성된 파이프라인 아키텍처를 채택한다. 각 단계는 LangGraph의 노드로 표현되며, 조건부 엣지를 통해 동적 분기가 이루어진다.

[시스템 파이프라인]

```
CloudWatch (1분) / Cost Explorer (1h)
  ↓ Boto3 수집기 → [raw metric buffer]
  ↓
STEP 1: Detection Agent (Z-score OR Isolation Forest)
  ↓ 이상 감지
STEP 2: Classification Agent (LLM) 유형 분류 + 임시조치
  ↓
STEP 3: Decision Agent (LLM) 후보 생성 → Score → Risk 판단
      ┌ Risk LOW → 즉시 실행
      └ Risk HIGH → 알림 → 60분 대기 → 자동 복구
  ↓
STEP 4: Action Agent (Boto3) AWS 리소스 직접 제어
  ↓
STEP 5: QA Agent SLA 체크 (실패 시 롤백 최대 2회)
  ↓ 통과
STEP 6: Logging Agent PostgreSQL Audit Log + Grafana 갱신
```

3.1.1 데이터 수집 및 버퍼링

CloudWatch와 Cost Explorer에서 리소스 별로 사용량 및 비용 데이터를 수집한다. 수집된 metric은 버퍼에 일시 저장되어 서로 다른 수집 주기를 맞추고, 이상 탐지를 위한 전처리에 활용된다.

3.1.2 Detection Agent

수집된 데이터를 기반으로 Z-score 또는 Isolation Forest 알고리즘을 적용하여 이상 여부를 판단한다. 이 단계에서는 정상 범위를 벗어난 패턴을 탐지하여 이상 신호를 생성한다.

3.1.3 Classification Agent

탐지된 이상 신호를 바탕으로 이상 유형을 분류한다. 이 과정에서 LLM을 활용하여 수집된 지표를 종합적으로 해석하고, 이상 원인을 추론한다. 예를 들어, 낭비형, 급증형, 공격형 등으로 구분하며, 각 유형에 따라 간단한 임시 조치를 취한다.

3.1.4 Decision Agent

분류 결과를 기반으로 여러 대응 방안을 생성하고, 각 방안에 대해 점수화 및 위험도 평가를 수행한다. 위험도가 낮은 경우 즉시 실행하며, 위험도가 높은 경우 사용자에게 알림을 전송해 사용자의 동의를 얻는다. 이때 일정 시간 이후에도 사용자의 응답이 없는 경우, 자동 복구를 수행한다.

3.1.5 Action Agent

선택된 대응 방안을 실제로 수행하는 단계로, Boto3를 활용하여 AWS 리소스를 직접 제어한다.

3.1.6 QA Agent

수행된 복구 작업이 정상적으로 이루어졌는지 검증한다. SLA를 기준으로 결과를 평가하며, 문제가 발생한 경우 최대 2회까지 롤백을 수행한다.

3.1.7 Logging Agent

전체 과정에서 발생한 이벤트와 결과를 기록한다. 로그는 PostgreSQL에 저장되며, Grafana를 통해 시각화되어 운영자가 시스템 상태를 모니터링할 수 있도록 한다. 저장된 로그는 시스템 유지보수 및 문제 분석에 활용된다.

3.2. 시스템 개발환경

- 백엔드 구현
 - Python 3.11 이상
- AI 프레임워크
 - LangChain : LLM 추상화 및 체인 구성 수행
 - LangGraph : 멀티 에이전트 기반 상태 머신 구조 구현
- LLM 환경(하이브리드 구조)
 - Ollama의 qwen:7b 모델
 - Gemini 2.5 Flash API
- 머신러닝 라이브러리
 - scikit-learn : Isolation Forest 기반 탐지 기능 구현
- 클라우드 리소스 제어
 - Boto3 : AWS SDK로 AWS 리소스를 제어
- 데이터 수집
 - CloudWatch
 - Cost Explorer
- 데이터베이스(Audit Log 저장)
 - PostgreSQL
- 이상 탐지 모니터링 및 시각화
 - Grafana
- 전체 서비스 구성

- Docker 및 Docker Compose : 각 구성 요소를 통합 실행

3.3 방법 구체화

3.3.1 Detection Agent (이상 탐지)

수집된 AWS 지표를 대상으로 두 가지 탐지 알고리즘을 병렬 적용하고 OR 양상블로 결합한다. 탐지 대상은 AWS 클라우드 환경 내 주요 리소스인 EC2, Lambda, S3, RDS이다. 각 리소스에서 CloudWatch를 통해 CPU 사용률, 네트워크 입출력, 호출 횟수, 비용 지표를 수집한다.

- **Z-score 기반 탐지 (단기 스파이크 대응)**

슬라이딩 윈도우(30분) 내에서 평균(μ)과 표준편차(σ)를 계산한 후, 각 시점의 값 x 에 대해 Z-score를 산출한다.

$$Z = (x - \mu) / (\sigma + \epsilon)$$

산출된 Z-score 가 임계값 $k(=3.0)$ 을 초과하는 경우 이상으로 판정한다.

이 기법은 단기적인 급격한 변화를 탐지하는데 효과적이며, 비용, 네트워크 입력, 호출 횟수 지표에 적용된다.

- **Isolation Forest 탐지 (다변량 복합 드리프트 대응)**

CPU 사용률, 네트워크 입출력, 비용, 호출 횟수 등 5개의 지표를 하나의 다변량 feature 벡터로 구성하여 Isolation Forest 모델에 입력한다. 모델이 산출한 이상 점수가 임계값 $\tau(=0.6)$ 을 초과할 경우 이상으로 판정한다.

이 기법은 여러 지표 간의 복합적인 변화 패턴을 반영하여 이상을 탐지할 수 있으며 24시간 주기로 재학습하여 데이터 분포 변화에 대응한다.

3.3.2 Classification Agent (유형 분류 및 임시조치)

탐지된 이상 신호에 대해 리소스 유형별로 원인을 분류한다. EC2·RDS는 좀비, 오버프로비저닝, 트래픽 폭증으로, Lambda는 무한루프 및 호출 폭증으로, S3는 비정상 트래픽 폭증으로, Auto Scaling은 EDoS 공격으로 인한 무분별한 스케일아웃을 주요 분류 대상으로 한다.

이러한 분류는 Rule-based 선처리를 통해 명확한 케이스를 우선 처리함으로써 LLM 호출 비용을 절감하고 분류 정확도를 향상시킨다. Rule-based 판단이 어려운 모호한 케이스에 한해 LLM을 활용하여 이상 유형을 분류한다. 이후 분류 결과에 따라 사전 정의된 임시 조치를 즉시 수행한다.

3.3.3 Decision Agent (액션 선택 및 Risk 판단)

분류된 이상 유형을 기반으로 가능한 대응 액션 후보를 생성하고, 각 액션에 대해 비용 절감, 서비스 영향도, 안정성을 종합적으로 고려하여 점수를 산출한다. 점수는 다음과 같은 가중 합 방식으로 계산된다.

$$Score(a) = w1 \times SavingRate(a) - w2 \times ImpactScore(a) + w3 \times StabilityScore(a)$$

- *SavingRate* : 비용 절감 효과
- *ImpactScore* : 서비스에 미치는 영향도
- *StabilityScore* : 시스템 안정성

기본 가중치는 비용 절감(0.5), 서비스 영향(0.3), 안정성(0.2)으로 설정하여 비용 효율성과 서비스 안정성 간의 균형을 고려한다.

계산된 점수를 기준으로 최적의 액션을 선택한 후, Risk Level에 따라 실행 방식을 결정한다. 위험도가 낮은 경우에는 즉시 실행하며, 중간 또는 높은 경우에는 사용자에게 알림을 전송하고 일정 시간(60분) 대기 후 자동으로 복구를 수행하는 에스컬레이션 절차를 따른다.

3.3.4 Action Agent (Boto3 리소스 제어)

Decision Agent에서 결정된 액션을 AWS Boto3 SDK를 통해 실제 리소스에 적용한다.

액션은 리소스 중지(stop), 변경(modify), 접근 차단(block), 상태 유지(no action)로 구분되며, 상황에 따라 적절한 대응을 수행한다. 또한 모든 액션 실행 전에는 리소스의 현재 상태를 스냅샷으로 저장하여 롤백에 활용한다.

3.3.5 QA Agent (검증 및 롤백)

액션 수행 이후 SLA 준수 여부를 기준으로 결과를 검증한다.

검증에 실패한 경우, 사전에 저장된 스냅샷을 기반으로 롤백을 수행한 후 재시도를 진행한다. 이러한 재시도는 최대 2회까지 허용되며, 2회를 초과할 경우 현재 상태를 유지하고 사용자(관리자)에게 알린다.

3.3.6 Logging Agent (Audit Log)

전체 에이전트 실행 과정과 결과를 기록하기 위해 PostgreSQL 기반의 Audit Log를 구성한다.

실행 이력은 agent_runs, agent_steps, action_log 테이블에 저장되며, 이를 통해 각 단계별 수행 결과와 상태를 추적할 수 있다.

또한 Grafana를 활용하여 비용 추이, 이상 탐지 빈도, 액션 성공률, Risk 분포, 단계별 지연 시간 등을 실시간으로 시각화함으로써 시스템 상태를 모니터링하고, 운영 및 유지보수에 활용한다.

4. 현실적 제약 사항 분석 결과 및 대책

4.1 LLM 환각(Hallucination)으로 인한 오판

LLM은 동일 입력에도 응답이 달라질 수 있으며, 존재하지 않는 리소스 ID를 언급하거나 실제 환경과 맞지 않는 과도한 비용 절감률을 반환할 가능성이 있다.

다음과 같은 방법으로 이러한 문제를 완화한다.

- 응답값 범위 클램핑 및 JSON 파싱 재시도

LLM이 반환하는 비용 절감률 값이 0.0~1.0 범위를 벗어나는 경우에는 허용 가능한 범위로 클램핑 처리한다. 또한 JSON 형식이 아닌 응답(마크다운 코드블록 포함, 불완전한 괄호 등)이 반환될 경우 최대 N회 재시도한다. 재시도 이후에도 파싱에 실패하면 해당 액션의 비용 절감률을 0.0으로 처리하여 NoAction이 선택되도록 유도한다. 이는 LLM 응답의 불안정성이 실제 AWS 리소스 조작으로 이어지는 것을 방지하기 위한 1차 방어선이다.

- 액션 후보 집합 검증 레이어

본 시스템은 이상 유형별로 허용된 액션 후보 집합이 명확히 정의되어 있다. 예를 들어 낭비형은 NoAction, Stop, Stop+Schedule, Resize만 허용되며, 급증형은 NoAction, Throttle, Block, ScaleDown만 허용된다. LLM이 이 집합에 없는 액션을 반환할 경우, 이를 유효하지 않은 응답으로 판단하고 자동으로 NoAction을 반환한다. 이는 LLM의 창의적 오답이 실제 리소스에 영향을 미치는 것을 막는 2차 방어선이다.

- temperature 고정

Gemini API 및 Ollama 호출 시 temperature=0.1로 고정하여 응답의 무작위성을 최소화한다. 완전한 결정론적 출력은 보장되지 않지만, 높은 temperature 대비 일관성 있는 응답을 유도할 수 있다.

4.2 AWS API 및 데이터 수집 관련 제약

다수 리소스를 동시에 모니터링하는 경우 CloudWatch get_metric_statistics, EC2 describe_instances 등의 API 호출이 AWS Rate Limit에 걸릴 수 있다. 특히 1분 주기 수집과 QA Agent의 5분 후 재조회가 겹칠 경우 Throttling Exception이 발생한다.

다음과 같은 방법으로 이러한 문제를 완화한다.

- Boto3 호출에 백오프 재시도 로직을 추가한다.
- CloudWatch Metrics Insights나 배치 조회 API를 활용해 호출 수를 줄인다.

4.3 일부 액션의 비가역성

ROLLBACK_ACTION은 대부분의 액션에 역방향 액션을 정의하고 있지만, 실제로는 완전한 원상 복구가 불가능한 경우가 있다. EC2 Resize의 경우 인스턴스를 중지하는 과정에서 EBS 외 임시 스토리지(인스턴스 스토어) 데이터가 유실된다. S3 Block(put_bucket_policy DENY_ALL) 롤백 시 원래 정책을 미리 백업해두지 않으면 복원이 불가능하다.

다음과 같은 방법으로 이러한 문제를 완화한다.

- boto3_response만으로는 원복에 필요한 정보가 부족하기에, Action Agent 실행 전 현재 상태 스냅샷(인스턴스 타입, 보안 그룹 규칙, 버킷 정책 원문)을 action_log 또는 별도 테이블에 저장한다.

5. 요약 및 기대 효과

5.1 요약

본 연구에서는 클라우드 비용 이상 징후를 단순 탐지하는 수준을 넘어, 탐지 – 분류 – 의사결정 – 복구 – 검증까지 전 과정을 자동화하는 Agentic AI 기반 자율 운영 시스템을 제안한다.

특히 Z-score와 Isolation Forest를 결합한 이상 탐지 구조와, LLM 기반 멀티 에이전트 파이프라인을 통해 기존의 정적 규칙 기반 시스템이 갖는 한계를 극복하고, 이상 상황에 대한 원인 해석과 대응 전략을 스스로 생성, 실행할 수 있는 구조를 구축하고자 한다.

이를 통해 기존의 “탐지 후 수동 대응” 방식 대비 대응 지연 시간을 크게 단축하고, 불필요한 클라우드 비용을 선제적으로 차단할 수 있는 가능성을 확인하였다. 또한 모든 의사결정 과정을 로그로 기록함으로써, LLM 기반 시스템의 한계로 지적되는 블랙박스 문제를 완화하고 설명 가능한 AI 운영 구조를 제시하였다.

다만, LLM의 환각 가능성, 일부 액션의 비가역성, AWS API 제약 등은 실제 운영 환경 적용 시 반드시 고려해야 할 요소이며, 본 과제에서는 이를 완화하기 위한 검증 및 안전 장치를 함께 설계하였다.

향후에는 강화학습 기반 액션 정책 최적화, 멀티 클라우드 환경 확장, 그리고 실제 프로덕션 환경에서의 장기적인 비용 절감 효과 검증을 통해 자율 클라우드 운영(AIOps)으로 확장 가능한 실용적 시스템으로 발전시킬 수 있을 것으로 기대된다.

5.2 기대효과

- 클라우드 비용 절감
낭비형 리소스 자동 중지 및 다운사이징으로 불필요한 과금을 방지한다.
- 대응 속도 향상
수동 대응 대비, 탐지부터 복구까지의 시간을 단축해 서비스 장애 시간을 최소화한다.
- 운영 부담 감소
반복적인 비용 모니터링 및 조치 업무를 자동화하여 엔지니어의 고부가가치 업무 집중이 가능하다.
- 보안 위협에 대한 즉각적인 대응
DDoS 의심 트래픽 급증 같은 위험 징후를 탐지해 보안 그룹 차단 등의 조치를 자동 실행한다.
- 확장성
본 시스템은 현재 AWS 환경을 기반으로 설계되었으나, 아키텍처 구조상 향후 확장 가능성을 내포하고 있다. LangGraph 기반의 에이전트 파이프라인은 특정 클라우드 벤더에 종속되지 않는

범용 구조로 설계되어 있으며, 리소스 제어 레이어(Boto3)를 GCP의 Cloud SDK나 Azure SDK로 교체하면 동일한 에이전트 로직을 타 퍼블릭 클라우드 환경에도 적용할 수 있다. 또한 Detection, Classification, Decision 등 각 에이전트가 독립적인 모듈로 분리되어 있어, 새로운 리소스 유형이나 탐지 시나리오가 추가되더라도 기존 파이프라인을 수정하지 않고 에이전트 단위로 기능을 확장할 수 있다.

- 학술적 기여
Agentic AI를 FinOps 영역에 적용한 사례 연구로서, 멀티 에이전트 롤백 메커니즘 설계 방법론을 제시한다.

6. 개발 일정 및 역할 분담

6.1 개발 일정

작업 항목	3월 3-4주	4월 1-2주	4월 3-4주차	5월 1-2주차	5월 3-4주차	6월 1-2주차	6월 3-4주차	7월 1-2주차	7월 3-4주차	8월 1-3주차	8월 4주차	9월
사전 학습 및 경쟁 기업 분석												
요구사항 분석 및 설계												
이상 / 복구 정의 및 시나리오별 대응 방안 구체화												
MVP 구현												
EC2, S3 리소스 이상 대응 구현												
Lambda, Auto scaling 이상 대응 구현												
Isolation Forest 학습 및 적용												
파이프라인 최종 설계/검증 및 컨테이너화												
유저 제공용 대시보드 설계												
최종 성능 평가 및 유지 보수												
최종 보고서 작성 및 발표												

6.2 역할 분담

팀원 3인의 역할을 다음과 같이 분담한다.

팀원	주요 구현 항목
허소영	LangGraph (Decision / flow) Boto3 액션 (Stop, Resize 등) 상태 스냅샷 저장 로직 롤백 로직 (QA Agent 일부)
박소영	LangGraph (Classification / 조건 분기) Logging Agent + PostgreSQL Grafana 대시보드 Z-score + Isolation Forest 앙상블
강지원	LangGraph 연결 및 조건부 엣지 일부 SLA 검증 비용 절감 검증 Classification Agent (Rule-based + LLM) 실험 시나리오 생성

Detection · Agentic AI 기반 클라우드 비용 이상 징후 탐지 및 자율 복구 시스템